

PARTE 1

Jantar dos Filósofos

Objetivo:

Mostrar o protocolo ingênuo que gera deadlock e uma solução que evita o impasse.

Protocolo Ingênuo:

Cada filósofo pega primeiro o garfo da esquerda e depois o da direita.

Isso pode gerar deadlock quando todos pegam simultaneamente o garfo esquerdo e ficam esperando o direito.

Por que ocorre o deadlock:

As 4 condições de Coffman são satisfeitas:

1. Exclusão mútua – um garfo por vez.
2. Manter-e-esperar – segura um garfo e espera outro.
3. Não-preempção – não dá pra tirar garfo de alguém.
4. Espera circular – cada filósofo espera o garfo do vizinho.

Solução utilizada:

Hierarquia de recursos: cada filósofo sempre pega primeiro o garfo de menor índice e depois o de maior índice.

Isso remove a espera circular → sem deadlock.

Pseudocódigos usados:

Protocolo ingênuo

Protocolo com hierarquia

PARTE 2

Condição de Corrida e Semáforo

Objetivo:

Mostrar que um contador acessado por várias threads sem controle apresenta race condition e depois corrigir usando Semaphore(1,true).

Versão sem controle:

- Múltiplas threads fazem count ++.
- A operação não é atômica.
- O valor final fica menor que o esperado ou seja perda de incrementos.

Versão com semáforo:

- Uso de Semaphore(1,true)
- Cada thread faz acquire(). incrementa e libera com release()
- Exclusão mútua garantida.
- Valor final correto.

O que foi observado:

Sem semáforo: inconsistência total.

Com semáforo: correto, porém mais lento.

PARTE 3

Deadlock com Threads em Java

Objetivo:

Criar um deadlock real e corrigi-lo.

Cenário implementado:

- LOCK_A e LOCK_B.
- Thread 1 pega A → tenta pegar B.
- Thread 2 pega B → tenta pegar A.
- Resultado: deadlock.

Condições de Coffman presentes:

1. Exclusão mútua.
2. Manter-e-esperar.
3. Não-preempção.
4. Espera circular.

Solução aplicada:

Hierarquia de locks: ambas as threads adquirem sempre na mesma ordem (A depois B).
Sem ciclo → sem deadlock.

Arquivos:

DeadlockDemo.java

DeadlockResolvido.java